DebtPulse — End-to-End Flow: Login → Report a Borrower Contact

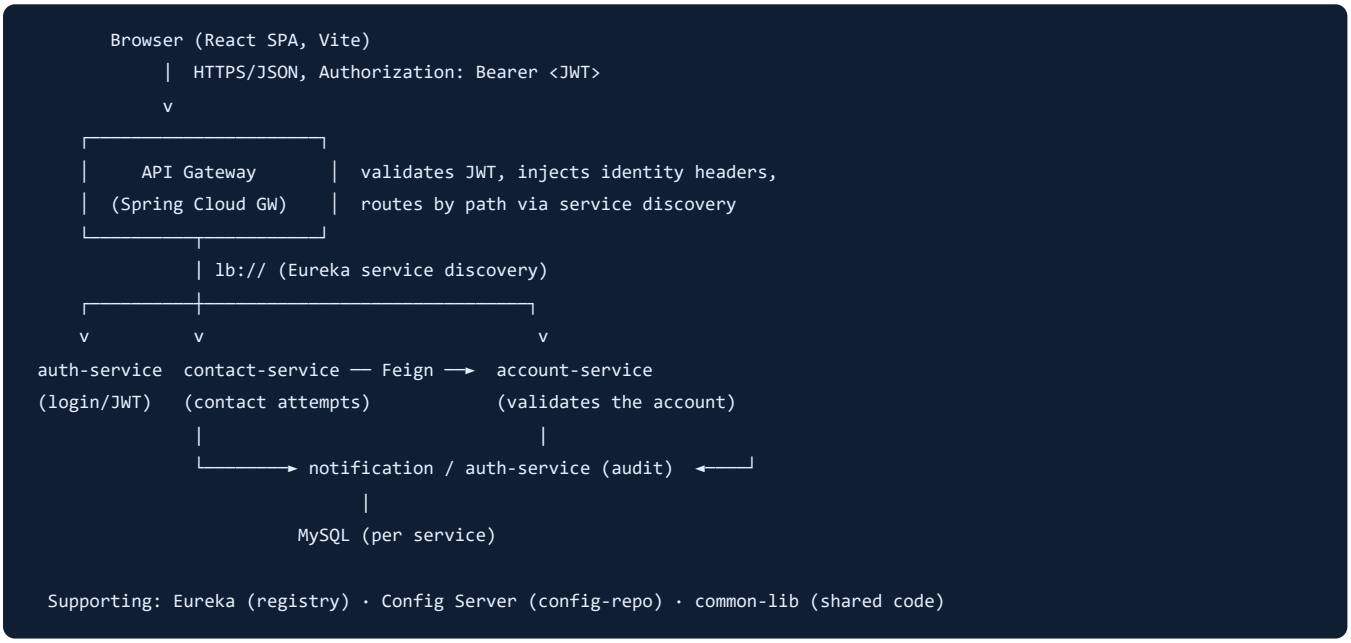
Purpose. A walkthrough of one complete request path through the whole system — from signing in, to submitting a "report" and getting a saved record back — with the exact data posted, what comes back, and every file the data passes through, in order.

Which flow? DebtPulse has no "incident" entity; its equivalent "report an event" action is a **Contact Attempt** — a collections agent logs the outcome of contacting a borrower (called, connected, promised to pay, etc.). It is the cleanest representative flow because it touches the entire stack: the React app, the API Gateway (JWT + routing), a business microservice (contact-service), a **cross-service call** to account-service, the database, and the central audit trail.

How to read this. Two flows are documented: **Flow A — Login** (get a token) and **Flow B — Log a Contact Attempt** (use the token to post a report and get the saved record back). Each has the code, the request/response, and an ordered list of the files the data flows through.

---

1. Architecture at a glance



- **API Gateway** is the single entry point and the one security choke point. The browser only ever talks to `/api/**` on the gateway.
- **Eureka** is the service registry; the gateway and Feign clients resolve `lb://service-name` to a live instance.
- **Config Server** serves shared config (`config-repo/*.yml`) to every service at boot.
- **common-lib** holds shared code every service reuses (JWT header filter, `AuthContext` , audit aspect, enums, DTOs).

---

2. Cross-cutting mechanics (know these first)

JWT + identity propagation. Login returns a short-lived **access token** (JWT, 3h) in the body and a long-lived **refresh token** as an `httpOnly` cookie. The SPA keeps the access token **in memory only** (never localStorage) and sends it as `Authorization: Bearer <jwt>` on every call. The gateway validates the JWT once and rewrites the request with **trusted identity headers** for downstream services:

```
X-Auth-UserId   X-Auth-Role   X-Auth-BranchId   X-Auth-Name
```

Downstream services never re-validate the JWT; they trust these headers (which the gateway strips from any inbound request first, so they cannot be spoofed) and rebuild the Spring Security context from them via `RoleBasedHeaderFilter` in common-lib:

```
// common-lib · RoleBasedHeaderFilter (per-request)
var authority = new SimpleGrantedAuthority("ROLE_" + role);   // role from X-Auth-Role
var authentication = new UsernamePasswordAuthenticationToken(userId, null, List.of(authority));
SecurityContextHolder.getContext().setAuthentication(authentication);
```

That is what makes `@PreAuthorize("hasRole('COLLECTIONS_AGENT')")` and `AuthContext.currentUserId()` work inside each service.

Service-to-service calls use OpenFeign over Eureka (`lb://account-service`) with a Resilience4j fallback, so one service can call another by name and degrade gracefully if it is down.

---

3. Flow A — Login

3.1 What the client sends / gets back

```
POST /api/auth/login
Content-Type: application/json

{ "email": "agent@dp.com", "password": "••••••••" }
```

```
200 OK
Set-Cookie: refresh_token=<opaque>; HttpOnly; Secure; SameSite=Strict; Path=/api/auth

{
  "message": "Login successful",
  "token": "<JWT access token>",
  "userId": "USR-002",
  "role": "COLLECTIONS_AGENT",
  "name": "Collections Agent",
  "branchId": "BR-01",
  "expiresIn": 10800
}
```

3.2 Frontend — submit and store

```
// LoginPage.jsx → calls the AuthContext
const res = await login(email, password);          // AuthContext.login

// auth/AuthContext.jsx
const { data } = await authApi.login(email, password);   // POST /api/auth/login
tokenStore.set(data.token);                                // access token kept in memory
setUser({ userId: data.userId, role: data.role, name: data.name, branchId: data.branchId, email });
```

```
// api/services.js
export const authApi = {
  login: (email, password) => api.post('/auth/login', { email, password }),
};

// api/client.js — axios instance; attaches the token to every later request
api.interceptors.request.use((config) => {
  if (accessToken) config.headers.Authorization = `Bearer ${accessToken}`;
  return config;
});
```

3.3 Gateway — open path (no token yet)

`/api/auth/` **is an** open path**, so the gateway's `JwtAuthenticationGlobalFilter` lets it through without a token and routes it to auth-service.

3.4 Backend — verify credentials, issue JWT

```
// auth-service · AuthController
@PostMapping("/login")
public ResponseEntity<AuthResponse> login(@Valid @RequestBody LoginRequest req) {
  AuthResponse res = authService.login(req.email(), req.password());
  return ResponseEntity.ok()
      .header(HttpHeaders.SET_COOKIE, refreshCookie(res.refreshToken(), ...).toString())  // httpOnly cookie
      .body(withoutRefreshToken(res));                                                     // token in body
}

// auth-service · AuthServiceImpl
User user = userRepo.findByEmail(email).orElseThrow(() -> new UnauthorizedActionException("Invalid credentials"));
if (isLocked(user)) throw ...;                       // lockout after repeated failures
if (!encoder.matches(password, user.getPasswordHash())) { registerFailedAttempt(user); throw ...; }
if (user.getStatus() != UserStatus.ACTIVE) throw ...;
return toResponse("Login successful", tokenService.issue(user));   // signs the JWT (subject=userId, role, branchId, name)
```

3.5 Back on the client

The access token is now in `tokenStore` (memory) and the `refresh_token` cookie is set by the browser. On a page reload the token is gone, so `AuthContext` silently calls `POST /api/auth/refresh` (cookie-based) to restore the session.

---

4. Flow B — Report a Borrower Contact (Log a Contact Attempt)

This is the "report" action. A logged-in collections agent submits what happened on a borrower contact and receives the persisted record (with a generated id and server timestamps).

4.1 What the client sends / gets back

```
POST /api/contacts
Authorization: Bearer <JWT>
Content-Type: application/json

{
  "accountId": "ACC-2026-000003",
  "channel": "CALL",
  "outcome": "CONNECTED",
  "notes": "Borrower promised to pay by month-end"
}
```

```
201 Created

{
  "contactId": "CON-2026-000007",
  "accountId": "ACC-2026-000003",
  "agentId": "USR-002",
  "contactDate": "2026-08-04T10:17:00",
  "channel": "CALL",
  "outcome": "CONNECTED",
  "notes": "Borrower promised to pay by month-end",
  "status": "LOGGED",
  "createdAt": "2026-08-04T10:17:00"
}
```

Note what the **server** fills in and returns: `contactId` (generated), `agentId` (from the JWT identity, not the client), `status = LOGGED`, `createdAt`.

4.2 Frontend — submit the report

```
// pages/contact/ContactWorkspace.jsx  (Contact Attempts tab → "Log contact")
<FormModal title="Log Contact Attempt" onSubmit={(v) => contactApi.create(v)} onSaved={reload}>
  <Field name="accountId" required />
  <Field name="channel" type="select" options={ENUMS.ContactChannel} required />
  <Field name="outcome" type="select" options={ENUMS.ContactOutcome} required />
  <Field name="notes" type="textarea" />
</FormModal>
```

```
// api/services.js
export const contactApi = {
  create: (body) => api.post('/contacts', body),
  list:   (params) => api.get('/contacts', { params }),
};
```

`FormModal` calls `contactApi.create(values)`; the axios interceptor attaches `Authorization: Bearer <token>`. On success it toasts and `reload()`s the list; on a validation/business error it maps the backend `fieldErrors` / `message` onto the form.

4.3 Gateway — authenticate + route

```
// api-gateway · JwtAuthenticationGlobalFilter (runs before routing)
Claims claims = jwtValidator.parse(token);                // verify signature + expiry
ServerHttpRequest mutated = request.mutate()
    .header("X-Auth-UserId", claims.getSubject())         // e.g. USR-002
    .header("X-Auth-Role",   claims.get("role", ...))     // COLLECTIONS_AGENT
    .header("X-Auth-BranchId", ...).header("X-Auth-Name", ...)
    .build();
return chain.filter(exchange.mutate().request(mutated).build());
// then routed via lb://contact-service to POST /api/contacts
```

4.4 Backend — controller (RBAC + validation)

```
// contact-service · ContactController
@PostMapping
@PreAuthorize("hasAnyRole('ADMIN','PORTFOLIO_MANAGER','COLLECTIONS_AGENT')")
public ResponseEntity<ContactAttemptDto> create(@Valid @RequestBody ContactAttemptRequest req) {
  return ResponseEntity.status(HttpStatus.CREATED).body(contactService.create(req));
}
```

- `RoleBasedHeaderFilter` (common-lib) has already rebuilt the security context from `X-Auth-*`, so `@PreAuthorize` can check the role.
- `@Valid` enforces the request contract: `accountId` not blank, `channel` / `outcome` not null.

4.5 Backend — service (business rules + cross-service call + persist + audit)

```
// contact-service · ContactServiceImpl
public ContactAttemptDto create(ContactAttemptRequest req) {
  // 1) The account must exist — cross-service call to account-service (Feign over Eureka)
  if (!accountClient.accountExists(req.accountId()))
      throw new BusinessRuleException("Account not found: " + req.accountId(), "ACCOUNT_NOT_FOUND");

  // 2) A collections agent may only act on accounts allocated to them (admins/managers exempt)
  assertAgentOwnsAccount(req.accountId());

  // 3) The agent id is taken from the authenticated caller, not trusted from the body
  String agentId = resolveAgentId(req.agentId());   // = AuthContext.currentUserId() for an agent

  // 4) Build + save the entity (id + createdAt generated)
  ContactAttempt entity = ContactAttempt.builder()
      .accountId(req.accountId()).agentId(agentId)
      .contactDate(req.contactDate() != null ? req.contactDate() : LocalDateTime.now())
      .channel(req.channel()).outcome(req.outcome()).notes(req.notes())
      .status(ContactStatus.LOGGED).build();
  ContactAttempt saved = repo.save(entity);

  // 5) Central audit (best-effort) + return the read DTO
  audit("CREATE", saved.getContactId());
  return mapper.toDto(saved);
}
```

The Feign client that makes the cross-service call:

```
// contact-service · AccountClient (Feign)
@FeignClient(name = "account-service", path = "/api/internal", fallback = AccountClientFallback.class)
public interface AccountClient {
  @GetMapping("/accounts/{id}/exists") boolean accountExists(@PathVariable String id);
  @GetMapping("/accounts/{id}")        AccountDto getAccount(@PathVariable String id);   // used by the ownership check
}
```

account-service answers on its **internal** API (service-to-service only, not exposed on the public gateway):

```
// account-service · InternalAccountController
@GetMapping("/accounts/{id}/exists")
public ResponseEntity<Boolean> exists(@PathVariable String id) {
  return ResponseEntity.ok(accountService.exists(id));
}
```

4.6 Backend — the return trip

`ContactAttempt` (entity) → `ContactAttemptMapper.toDto()` → `ContactAttemptDto` (JSON) → 201 back through the gateway → axios resolves the promise → `FormModal` toasts success and reloads the table.

```
// contact-service · ContactAttemptMapper
public ContactAttemptDto toDto(ContactAttempt c) {
  return new ContactAttemptDto(c.getContactId(), c.getAccountId(), c.getAgentId(),
      c.getContactDate(), c.getChannel().name(), c.getOutcome().name(),
      c.getNotes(), c.getStatus().name(), c.getCreatedAt());
}
```

4.7 Sequence (report flow)

```
Agent ─► ContactWorkspace ─► contactApi.create ─► [axios +Bearer]
      ─► API Gateway (validate JWT, add X-Auth-*) ─► lb://contact-service
      ─► ContactController @PreAuthorize + @Valid
      ─► ContactServiceImpl.create
            ├─► account-service GET /internal/accounts/{id}/exists   (Feign)
            ├─► account-service GET /internal/accounts/{id}          (ownership check)
            ├─► repo.save(ContactAttempt)                            (MySQL)
            └─► auth-service audit log                               (best-effort)
      ◄─ 201 ContactAttemptDto ◄─ mapper.toDto ◄─ saved entity
Agent ◄─ toast "saved" + list reloads
```

5. Endpoint reference

Auth + Contact (this flow)

Method	Path	Auth	Purpose
POST	`/api/auth/login`	open	Credentials → access token (body) + refresh cookie
POST	`/api/auth/refresh`	cookie	Silent session restore → new access token
POST	`/api/auth/logout`	cookie	Revoke session, clear cookie
POST	`/api/contacts`	ADMIN / PM / AGENT	Log a contact attempt (the "report")
GET	`/api/contacts`	ADMIN / PM / AGENT	List/filter contact attempts (accountId, agentId, …)
GET	`/api/contacts/{id}`	ADMIN / PM / AGENT	Fetch one attempt
GET	`/api/internal/accounts/{id}/exists`	service-to-service	account-service existence check (Feign)

Other module entry points (same pattern: gateway → service → DB)

Module	Representative endpoints
Portfolio	`GET/POST /api/accounts`, `POST /api/accounts/import/csv`, `POST /api/allocations/execute`
Contact	`/api/contacts`, `/api/ptp`, `/api/borrower-contacts`
Field	`/api/visits`, `/api/asset-verifications`
Settlement	`/api/settlements`, `/api/restructuring`
Legal	`/api/legal/cases`, `/api/legal/hearings`, `/api/legal/orders`
Analytics	`/api/analytics/dashboard`, `/api/analytics/reports`
Notifications	`/api/notifications`, `/api/notifications/unread-count`

6. File-flow summary (in the order data travels)

Flow A — Login

#	File	Layer	What to know
1	`frontend/src/pages/LoginPage.jsx`	FE view	The login form; calls `AuthContext.login`.
2	`frontend/src/auth/AuthContext.jsx`	FE state	Calls `authApi.login`, stores token + user, exposes `useAuth()`.
3	`frontend/src/api/services.js`	FE API	`authApi.login` → `POST /api/auth/login`.
4	`frontend/src/api/client.js`	FE HTTP	Axios instance; holds the in-memory token; `withCredentials` for the refresh cookie.
5	`api-gateway/.../JwtAuthenticationGlobalFilter.java`	Gateway	`/api/auth/**` is an open path → passes through.
6	`auth-service/.../controller/AuthController.java`	BE ctrl	`login()` → sets refresh cookie, returns token.
7	`auth-service/.../service/impl/AuthServiceImpl.java`	BE svc	Verifies credentials, lockout, status; issues JWT.
8	`config-repo/auth-service.yml` + `application.yml`	Config	JWT secret/expiry, DB, Eureka (shared).

Flow B — Log a Contact Attempt (the report)

#	File	Layer	What to know
1	`frontend/src/pages/contact/ContactWorkspace.jsx`	FE view	"Log contact" form; calls `contactApi.create`, reloads on success.
2	`frontend/src/components/FormModal.jsx`	FE view	Generic create/edit modal; maps backend `fieldErrors` / `message` to inputs.
3	`frontend/src/api/services.js`	FE API	`contactApi.create` → `POST /api/contacts`.
4	`frontend/src/api/client.js`	FE HTTP	Attaches `Authorization: Bearer <token>`; normalises errors; 401→silent refresh+retry.
5	`api-gateway/.../JwtAuthenticationGlobalFilter.java`	Gateway	Validates JWT, injects `X-Auth-*`, routes `lb://contact-service`.
6	`common-lib/.../security/RoleBasedHeaderFilter.java`	BE (shared)	Rebuilds Spring Security context from `X-Auth-*` so `@PreAuthorize` works.
7	`contact-service/.../controller/ContactController.java`	BE ctrl	`@PreAuthorize` role check + `@Valid` request; delegates to the service.
8	`contact-service/.../dto/request/ContactAttemptRequest.java`	BE DTO	Inbound contract (`accountId`, `channel`, `outcome`, `notes`).
9	`contact-service/.../service/impl/ContactServiceImpl.java`	BE svc	Business rules: account exists, ownership, agent-from-JWT, save, audit.
10	`contact-service/.../feign/AccountClient.java`	BE client	Cross-service call to account-service (exists + ownership).
11	`account-service/.../controller/InternalAccountController.java`	BE ctrl	Answers the internal existence/lookup call.
12	`contact-service/.../entity/ContactAttempt.java`	BE entity	Persisted row; generates `CON-…` id, `createdAt`, default `status=LOGGED`.
13	`contact-service/.../repository/ContactAttemptRepository.java`	BE data	Spring Data JPA save/query → MySQL.
14	`contact-service/.../mapper/ContactAttemptMapper.java`	BE map	Entity → `ContactAttemptDto` (the JSON that returns).
15	`contact-service/.../dto/response/ContactAttemptDto.java`	BE DTO	Outbound contract returned to the client (201).
16	`contact-service/.../feign/AuthClient.java`	BE client	Best-effort central audit write.

7. How to explain it to each audience

To the Project Manager (business view). "When an agent signs in, the system proves who they are and hands them a secure pass. When they log what happened on a borrower call, the request goes through one guarded front door (the gateway), which checks the pass and stamps who they are. The contact service then confirms the account is real and actually assigned to that agent, records the interaction with a unique reference, writes an audit entry for compliance, and hands back the saved record — which appears instantly in their list. Nothing bypasses the front door, and agents can only act on their own accounts."

To the Senior Developer (technical view). "Stateless JWT auth: access token in memory, refresh token as an httpOnly cookie. The gateway is the only JWT validator; it strips and re-injects `X-Auth-*` identity headers, which `RoleBasedHeaderFilter` turns back into a `SecurityContext` in each service, so `@PreAuthorize` is enforced service-side. `POST /api/contacts` runs `@Valid` + RBAC, then the service does a Feign call to account-service (`/api/internal`) for existence and an ownership guard, persists via Spring Data JPA, and fires a best-effort audit. The response is a mapped DTO — the server owns `agentId` (from the token), the generated id, timestamps and `status`. Feign runs over Eureka with Resilience4j fallbacks, and shared concerns live in common-lib."

Generated for the DebtPulse project. Flow documented: Login → Log a Contact Attempt (borrower-interaction report).